

SPoCK: Sequential Proofs of Complete Knowledge

Antonio Giulio D’Antona¹, Charlotte Hoffmann², and Krzysztof Pietrzak²

¹University of Pisa

`antoniogiuliodantona@gmail.com`

²Institute of Science and Technology Austria

`{charlotte.hoffmann, pietrzak}@ista.ac.at`

Abstract

A proof of knowledge certifies that the prover “knows” a secret. This property is established by the existence of an extractor that, given access to the prover, can extract the secret. Unfortunately, this does not imply that a single entity has access to the secret in the clear, as it may be secret-shared among several parties and the proof computed using multi-party computation (MPC), or the secret may be embedded in a trusted execution environment (TEE) that only allows limited access to it.

Often, the ability to encumber a secret in this way breaks the security of a system; vote selling in e-voting schemes is one example. To address this, two recent papers introduced the notions of “Proofs of Complete Knowledge” (PoCK) [Kelkar et al., CCS’24] and “Individual Cryptography” [Dziembowski et al., CRYPTO’23]. While the goals and constructions in those works differ, both rely on the same key idea to prevent encumbrance. To compute a proof, the prover must evaluate a hash function on a huge number of inputs. One then assumes that only a fraction of those can be encumbered, while for the rest the prover must know the inputs in the clear, and these clear inputs are sufficient to extract the secret. Computing a huge number of hashes is a challenge even for an honest prover (who has the secret in the clear), so those works suggest using outdated Bitcoin mining hardware to make the scheme practical.

By forcing the prover to evaluate the hashes in a *sequential* manner, we could get meaningful security against encumbrance with far fewer hashes, especially against MPC, where round complexity is a major bottleneck.

As a concrete instantiation of this idea, we define and construct **Sequential Proofs of Complete Knowledge (SPoCK)**. Our construction uses the PoCK of Kelkar et al. (which are based on Fischlin’s straight-line extractable proofs of knowledge). The computation of this PoCK is then embedded into the computation of the Proof of Sequential Work from [Cohen&Pietrzak, EUROCRYPT’18]. SPoCK thus have the potential to

enable complete-knowledge or individual-cryptography primitives where honest parties can use standard hardware. We also propose a variant of this scheme that requires a large amount of memory throughout the evaluation, providing better security also against TEEs.

1 Introduction

Shortly after zero-knowledge proofs [GMR85], which allow one to prove that some NP statement is true, zero-knowledge proofs *of knowledge* (ZK-PoK) [GMW86] were introduced. In these protocols, the proof also certifies that the prover knows an actual witness; that is, for some NP relation $\mathcal{R}_L$ for a language L , given $x \in L$, the proof demonstrates knowledge of a w such that $(x, w) \in \mathcal{R}_L$. To prove “knowledge”, one must provide an extractor Ext that can extract a witness from any, potentially malicious, prover who can make the verifier accept with sufficiently high probability.

ZK-PoK are interesting even for languages where membership in L is trivial. For example, consider the discrete logarithm relation

$$((g, h), x) \in \mathcal{R}_{\text{DL}} \iff g^x = h$$

in a prime-order group. Here every pair (g, h) of group elements (with g not the identity) is in the language, but finding x can be hard.

ZK-PoKs have found many applications in cryptography, where one typically thinks of (x, w) as a public-key/secret-key pair. Some languages (like discrete log) allow for particularly elegant ZK-PoKs called Σ -protocols, which we define in §2. Informally, these are three-move public-coin protocols [FS87, FFS87] that satisfy the “special honest-verifier zero-knowledge” and “special soundness” properties. The first means that there exists a simulator Sim that, given any second message (the challenge) c , can sample an entire transcript (a, c, z) with the same distribution as a real transcript. The special soundness property states that from any two accepting transcripts (a, c, z) and (a, c', z') with the same first, but different second message, one can extract a witness.

A Σ -protocol can be turned into a non-interactive ZK-PoK (NI-ZK-PoK) in the random-oracle model using the Fiat–Shamir transform [FS87]. In this transform, the challenge in the second message is not sampled at random by the verifier, but is instead the output of the random oracle on the instance and the first message (we give an example in §2.2). To argue knowledge soundness of this transformation, one typically assumes that the extractor can program the random oracle and rewind the prover. Such a rewinding extractor is problematic, for example for composition (in particular UC [Can01] or when recursively composing zero-knowledge proofs [BCCT13]) or in the quantum setting [Unr12].

A NI-ZK-PoK allows for *straight-line extraction* if the extractor does not require rewinding. A transform from Σ -protocols into straight-line extractable NI-ZK-PoKs in the random-oracle model was proposed by Pass [Pas03], relying on cut-and-choose techniques. A more efficient transformation using “proofs of work” was later proposed by Fischlin [Fis05]; we review his transform in §2.3.2.

These transformations also do not require the extractor to program the random oracle; the extractor only needs to observe the queries made by the prover to the random oracle $\mathcal{H}$.

Recently, two papers on “proofs of complete knowledge” by Kelkar et al. [KBD⁺24], and “individual cryptography” by Dziembowski, Faust, and Lizeurj [DFL23], argue that even straight-line extraction is not always sufficient.

As an example, consider vote selling. A party might hold a secret that is necessary for voting, and also be inclined to sell its voting rights, but not give away the secret, for example because it can be used to retrieve collateral on a blockchain. This party could “encumber” its secret, for example by secret-sharing it among several parties that it believes will not collude to steal its collateral, but are willing to engage in a multiparty computation (MPC) to sign votes. Alternatively, this party could use a trusted execution environment (TEE), such as Intel SGX, that contains the secret and will sign votes, but will not help in retrieving the collateral. We refer to [KBD⁺24, DFL23, DFLM24, BDF⁺25, KGP⁺25, AFK⁺25] for more examples.

Both papers [KBD⁺24, DFL23] suggest the same high-level idea to prevent encumbrance in the random-oracle model: construct schemes where the computation that depends on the secret (say, computing a signature) must make a huge number q of queries to the random oracle $\mathcal{H}$. Choose q sufficiently larger than the number s of queries that one believes cannot be evaluated in an encumbered way, and ensure that the secret can be efficiently extracted from any $q - s$ queries. Formally, knowledge soundness is modeled by not giving the extractor all q queries to $\mathcal{H}$ made by an adversary (as in straight-line extraction), but only $q - s$ of them, where the s omitted queries are chosen by the adversary. In [DFL23], and also in this work, the adversary may additionally split those $q - s$ queries into p sets, and extraction must succeed from at least one of them. This models that in an MPC computation with p parties, at least one party must know the secret in the clear.

The parameter s represents an upper bound on the number of hash queries we believe an adversary could evaluate in an encumbered way, meaning that no single party learns the entire input to the query in the clear. Ideally, s should be large, so [KBD⁺24, DFL23] suggest using outdated Bitcoin ASICs, which can evaluate the SHA256 hash function extremely efficiently, for this purpose.

In this work, we leverage the observation—already discussed in [DFL23]—that encumbrance, especially via MPC, becomes particularly difficult when the computation is highly sequential, since this significantly increases the round complexity of practical MPC protocols.

We revisit the construction of zero-knowledge proofs of complete knowledge (ZK-PoCK) from [KBD⁺24]. Their construction essentially applies the Fischlin transform to a Σ -protocol, but uses a much higher difficulty for the underlying proof of work in order to force the prover to make a huge number of queries.

We adapt their definition to obtain zero-knowledge *sequential* proofs of complete knowledge (ZK-SPoCK), where we not only require that a prover makes a large number of $\mathcal{H}$ queries, but additionally that these queries must be made *sequentially*. In practice, this allows us to use a much more conservative bound

on the number of encumbered queries s , leading to more practical constructions that can potentially be executed on standard hardware rather than Bitcoin ASICs.

We construct a ZK-SPoCK by embedding the ZK-PoCK from [KBD⁺24] into the computation of the proof of sequential work (PoSW) from [CP18]. Our scheme can be made non-interactive and turned into a signature scheme using the Fiat–Shamir transform.

While sequentiality is a powerful tool to prevent encumbrance via MPC, it is less effective against TEEs. To address this, in §7 we consider a variant of our ZK-SPoCK using the proof of sequential work from [MMV13] instead of [CP18], thereby forcing the (honest or malicious) prover to perform not only a long sequential computation, but also to use a large amount of memory during the computation, which is something TEEs struggle with.

2 Σ -protocols

A zero-knowledge proof of knowledge [GMW86] (zkPoK) for an NP language L given by a relation $\mathcal{R}_L$ (i.e., $x \in L \iff \exists w : \mathcal{R}_L(x, w) = 1$) is an interactive protocol between a prover $\mathsf{P}(x, w)$ and a verifier $\mathsf{V}(x)$ where the prover convinces the verifier that they know a witness w' such that $\mathcal{R}_L(x, w') = 1$.

The zero-knowledge-property is captured by the existence of an efficient simulator who can sample transcripts that are indistinguishable from a real interaction, while the proof of knowledge property is captured by the existence of an efficient extractor, which, given access to a prover that can make the verifier accept, will extract a witness.

Some languages, like discrete logarithms $\mathcal{R}_{\text{DL}}((g, h), w) = 1 \iff g^w = h$ where g is a generator of a prime order group, allow for particularly elegant zkPoKs called Σ -protocols (or sigma-protocols) [FS87, FFS87].

Definition 1. A Σ -protocol (sigma-protocol) for a relation $\mathcal{R}$ is a three-move public-coin protocol defined by three PPT algorithms $(\mathsf{P}_\Sigma, \mathsf{V}_\Sigma, \mathsf{Sim}_\Sigma, \mathsf{Ext}_\Sigma)$.

- $\mathsf{P}_\Sigma(x, w) \rightarrow (a, \text{st})$ on input an instance and witness $(x, w) \in \mathcal{R}$ outputs a value (called commitment) a and state st , a is send to V .
- $\mathsf{V}_\Sigma(x, a) \rightarrow c$ then samples a random challenge $c \leftarrow^s \mathcal{C}$ from a challenge space $\mathcal{C}$.
- $\mathsf{P}_\Sigma(x, w, c, \text{st}) \rightarrow z$ computes the answer z .
- $\mathsf{V}_\Sigma(x, a, c, z) \rightarrow \{\text{acc}, \text{rej}\}$ decides whether to accept the transcript.

A Σ -protocol has the following properties:

perfect completeness: $\mathsf{V}_\Sigma(x)$ always accepts when interacting with an honest prover, i.e., for any $(x, w) \in \mathcal{R}$

$$\Pr[\mathsf{V}_\Sigma(x, a, c, z) = \text{acc} \mid (a, \text{st}) \leftarrow \mathsf{P}_\Sigma(x, w); c \leftarrow^s \mathcal{C}; z \leftarrow \mathsf{P}_\Sigma(x, w, c, \text{st})] = 1$$

special soundness: From two accepting transcripts with the same first, but different 2nd message, one can efficiently extract a witness, i.e.,

$$\text{Ext}_\Sigma(x, (a, c, z), (a, c', z')) \rightarrow w \text{ where } (x, w) \in \mathcal{R}$$

special honest-verifier zero-knowledge: For any $(x, w) \in \mathcal{R}$, the simulator Sim_Σ can sample a transcript that is (computationally or statistically) close to a real interaction

$$\left\{ (a, c, z) \mid \begin{array}{l} c \leftarrow \mathcal{C} \\ (a, z) \leftarrow \text{Sim}_\Sigma(x, c) \end{array} \right\} \approx_{c/s} \left\{ (a, c, z) \mid \begin{array}{l} (a, \text{st}) \leftarrow \text{P}_\Sigma(x, w) \\ c \leftarrow \mathcal{C} \\ z \leftarrow \text{P}_\Sigma(x, w, c, \text{st}) \end{array} \right\}.$$

2.1 Schnorr's Protocol for DL.

Let $\mathbb{G}$ be a prime order cyclic group of order q and g some generator. Schnorr's protocol is a Σ -protocol for the language of discrete logarithms where $((g, h), w) \in \mathcal{R}_{\text{DL}}$ if $h = g^w$.

- $\text{P}((g, h), w) \rightarrow (a = g^r, \text{st} = r)$ where $r \leftarrow \mathbb{Z}_q$
- $\mathcal{C} = \mathbb{Z}_q$
- $\text{P}((g, h), w, c, r) \rightarrow (z = r + c \cdot w)$
- $\text{V}((g, h), a, c, z) = \text{acc} \iff g^z = a \cdot h^c$

2.2 Removing interaction: Fiat-Shamir

As Σ -protocols are public-coin, i.e., the verifier only sends a uniformly random challenge, one can make the protocol non-interactive by deriving the challenge as a hash $\mathcal{H}(x, a)$ of the instance and the first prover message, this is known as the Fiat-Shamir transform [FS87]. For the discrete log relation $\mathcal{R}_{\text{DL}}$, such a non-interactive zero-knowledge proof of knowledge (NI-ZK-PoK) is computed as

$$\begin{aligned} \text{P}((g, h), w) &\rightarrow ((g, h), a, c, z) \quad \text{where} \\ r &\leftarrow \mathbb{Z}_q, a = g^r, c := \mathcal{H}((g, h), a), z = r + c \cdot w \end{aligned} \tag{1}$$

The verification is as in the interactive protocol, but one must additionally checks that c was correctly derived

$$\text{V}((g, h), a, c, z) = \text{acc} \iff g^z = a \cdot h^c \text{ and } c = \mathcal{H}((g, h), a)$$

Schnorr's signature scheme is basically this proof system, except that for signing a message msg , one would also add m to the hashed string, i.e., $c := \mathcal{H}(\text{msg}, (g, h), a)$.

2.3 Knowledge soundness: rewinding, straight line and complete

In an (interactive) Σ -protocol we are convinced that a successful prover “knows” a witness w , $(x, w) \in \mathcal{R}$, as from any prover who is sufficiently successful, we can extract a witness. Formally, there’s a knowledge extractor Ext which, given access to a prover $\tilde{P}$ that makes the verifier accept with some sufficiently high probability, can extract a witness. For this, Ext can simply run $\tilde{P}$, providing a random challenge c , and hopefully this way extract an accepting transcript (a, c, z) . Then *rewind* $\tilde{P}$ to the point where it received c , and send another random challenge $c' \neq c$ instead. If this gives an accepting transcript (a, c', z') , by special soundness, the extractor can compute a witness.

2.3.1 Fiat-Shamir (Rewinding Extractor)

Once we make the protocol non-interactive using Fiat-Shamir, the challenge c is computed as a hash of $\mathcal{H}(x, a)$, and thus rewinding no longer works. We can still show knowledge soundness in this case, but now have to rely on the random-oracle heuristic, where $\mathcal{H}$ is modeled as a random function which all parties can only access through black-box queries. Concretely, Ext can still extract a witness if (1) we allow the extractor to observe all the queries made by $\tilde{P}$ (2) to rewind $\tilde{P}$ and (3) also to *program* $\mathcal{H}$ (i.e., Ext can choose the outputs for $\mathcal{H}$ on some chosen inputs).

2.3.2 Fischlin’s transform (Straight-Line Extractor)

Fischlin [Fis05] proposed an alternative way to transform a Σ -protocol into a NI-ZK-PoK. While the NI-ZK-PoK from Fischlin’s transformation is less efficient than using Fiat-Shamir, it has a straight-line knowledge extractor (i.e., no rewinding is necessary), moreover the extractor does not need to program the random oracle. The extractor can compute a witness just given the inputs to the random-oracle queries made in one execution of the prover.

The idea is to let the prover hash entire valid transcripts (a, c, z) of the Σ -protocol in such a way, that a successful prover must almost certainly hash at least two tuples $(a, c, z), (a, c', z')$ with the same first message, but different challenge. The extractor, who can observe those inputs, can then extract a witness from any such pair.

Consider a Σ -protocol with prover and verifier (P_Σ, V_Σ) for a relation $\mathcal{R}$. Applying Fischlin’s transform we get a NI-ZK-PoK with prover and verifier (P, V) as outlined below. The transform is parametrized by the number of executions M and a proof-of-work difficulty d . We get roughly $\approx M \cdot d$ bits “knowledge soundness security” (so $M = d = 16$ is a good choice if we aim for 256 bit security). The proof size and verifier complexity scales like M , while the prover complexity scales like $M \cdot 2^d$.

Below we outline the NI-ZK-PoK $(P_F, V_F, \text{Sim}_F, \text{Ext}_F)$ we get by applying Fischlin’s transform to a Σ -protocol $(P_\Sigma, V_\Sigma, \text{Sim}_\Sigma, \text{Ext}_\Sigma)$. We sketch a some-

what simpler version where the prover runs in *expected* time $M \cdot 2^d$, while the running time in Fischlin [Fis05] is strictly polynomial, but there's an exponentially small completeness error.

Random Oracle $\mathcal{H}$ and Security Parameter λ . All algorithms $(P_F, V_F, \text{Sim}_F, \text{Ext}_F)$ get access to a random oracle $\mathcal{H} : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$, where throughout λ denotes a statistical security parameter. Ext doesn't really need the random oracle (it just gets to see some of the queries made to it), while Sim_F in some protocols is even allowed to program $\mathcal{H}$.

ρ for Salting. The prover $P_F^{\mathcal{H}}$ will get as input the instance and witness $(x, w) \in \mathcal{R}$, and we additionally allow a value $\rho \in \{0, 1\}^\lambda$. This value will then be used as prefix to all $\mathcal{H}$ queries made, where to save on notation we define $\mathcal{H}_\rho(\cdot) \stackrel{\text{def}}{=} \mathcal{H}(\rho, \cdot)$. One function of ρ is domain separation, by sampling a fresh random $\rho \leftarrow \{0, 1\}^\lambda$ before invoking the prover, we can basically assume that $\mathcal{H}_\rho$ behaves like a freshly sampled random oracle even if the malicious prover did some polynomially bounded precomputation [DGK17].

Definition of Fischlin's transform. The prover $P_F^{\mathcal{H}}(\rho, (x, w))$ where $(x, w) \in \mathcal{R}$

- Runs $(a_i, \text{st}_i) \leftarrow P_\Sigma(x, w)$ M times to get a vector $\mathbf{a} = (a_0, \dots, a_{M-1})$ of first messages.
- For each $i = 0, \dots, M-1$, $P_F^{\mathcal{H}}$ computes transcripts $(a_i, j, z_{i,j})$ (where $z_{i,j} = P_\Sigma(x, w, j, \text{st}_i)$) for challenges $j = 0, 1, 2, \dots$ until it hits a tuple $\pi_i = (a_i, j, z_{i,j})$ where $\mathcal{H}_\rho(x, i, \mathbf{a}, \pi_i)$ starts with d zeros. The proof is $\pi = \{\pi_i\}_{i=0}^{M-1}$.

The verifier $V_F^{\mathcal{H}}(\rho, x, \pi)$ where $\pi = \{\pi_i\}_{i=0}^{M-1}, \pi_i = (a_i, c_i, z_i)$.

- Checks that V_Σ accepts all π_i and that for all $i = 0, \dots, M-1$, $\mathcal{H}_\rho(x, i, \mathbf{a}, \pi_i)$ starts with d zeros. $V^{\mathcal{H}}$ accepts iff all checks pass.

The straight line extractor Ext_F gets as input the instance x and the queries $\mathcal{Q}$ made by P_F . By the following claim, for sufficiently large M, d , Ext_F will almost certainly be able to extract a witness whenever $\tilde{P}_F$ outputs a valid proof.

Claim 1 (Straight-line Extractor for Fischlin's transform). *Consider any $\tilde{P}_F$ and let $\mathcal{Q}$ denote the queries $\tilde{P}_F$ makes to its oracle $\mathcal{H}$, then*

$$\Pr_\rho[V_F^{\mathcal{H}}(\rho, x, \pi) = \text{acc} \wedge (x, w) \notin \mathcal{R} \mid \pi \leftarrow \tilde{P}_F^{\mathcal{H}}(\rho, (x, w)) ; w \leftarrow \text{Ext}_F(x, \mathcal{Q})] \leq \frac{|\mathcal{Q}|}{2^{M \cdot d}}.$$

Proof. To find a proof, $\tilde{P}_F$ must choose some $\mathbf{a} = (a_0, \dots, a_{M-1})$, and then, for each $i \in [0, M-1]$, find a valid proof $\pi_i = (a_i, j, z_{i,j})$ where $\mathcal{H}_\rho(x, i, \mathbf{a}, \pi_i)$ starts with d zeros. If for any i it queries two different $(a_i, j, z_{i,j}), (a_i, k, z_{i,k}), k \neq j$,

we can extract a witness. To avoid this, $\tilde{\mathbf{P}}_F$ must be “lucky” with all M queries and hit a tuple where the hash starts with d ones the first try. For a fixed $\mathbf{a}$ this happens with probability $(2^{-d})^M$. The bound follows by taking the union bound over the at most $|\mathcal{Q}|$ different $\mathbf{a}$ vectors $\tilde{\mathbf{P}}_F$ tries out. $\square$

2.3.3 Proofs of Complete Knowledge

Above, we let Ext_F observe all the $\mathcal{H}$ queries made by $\tilde{\mathbf{P}}_F^{\mathcal{H}}$. In a proof of complete knowledge, we want Ext_F to succeed even if $\tilde{\mathbf{P}}_F^{\mathcal{H}}$ can make a large number s of “slow” queries that are not given to Ext_F to capture “encumbered queries” (done using an MPC protocol, or a TEE), the remaining queries are called “fast”. Moreover we want to capture a distributed prover made up of up to p parties.

To model this, we allow the malicious prover to split all the fast queries into p parts, and require that Ext_F can extract the witness from at least one of those parts, thus capturing that at least one of those distributed adversaries must know the secret.

Concretely, we model our adversary $\tilde{\mathbf{P}}_{F,s,p}^{\mathcal{H}}$ as a monolithic adversary but we let it choose which s queries to remove from $\mathcal{Q}$, and how to split the remaining queries into (potentially empty) sets $\mathcal{Q}_1, \dots, \mathcal{Q}_p$. This adversary breaks complete-knowledge soundness if we can’t extract a witness from any of $\mathcal{Q}_i$ ’s. To compute the complete-knowledge error for Fischlin’s transform we must, for any malicious prover $\tilde{\mathbf{P}}_{s,p}$, bound

$$\Pr_{\rho} \left[\begin{array}{l} \mathbf{V}^{\mathcal{H}}(\rho, x, \pi) = \text{acc} \\ \text{and } \forall i \in [p] : (x, w) \notin \mathcal{R} \end{array} \middle| \begin{array}{l} (\pi, \mathcal{Q}_1, \dots, \mathcal{Q}_p) \leftarrow \tilde{\mathbf{P}}_{s,p}^{\mathcal{H}}(\rho, (x, w)) \\ \text{where } \forall i \in [p] : \mathcal{Q}_i \subseteq \mathcal{Q} \\ \text{and } |\cup_{i=1}^p \mathcal{Q}_i| \geq |\mathcal{Q}| - s \\ w_i \leftarrow \text{Ext}(x, \mathcal{Q}_i) \end{array} \right]$$

This probability is just the probability that we solve the M proofs of work, each of difficulty 2^{-d} using our s slow queries (we get an additional $p \cdot M$ queries as the adversary is distributed, but as in practice $s \gg p \cdot M$, we can ignore this). More formally, this is the probability that, when sampling s variables ($s + p \cdot M$ to be precise), each being 1 with probability 2^{-d} , we see at least M 1’s.

2.3.4 A numerical example

To choose parameters, we need to make an assumption on s , say we believe no adversary can make $s = 2^{30}$ (around a billion) *encumbered* queries in the time from receiving the beacon ρ to outputting the proof. Now, we could set $M = 16 = 2^4$ and $d = 34$, and the bound becomes roughly 2^{-116} (by a Chernoff bound).

Looking ahead, for the *sequential* proof of complete knowledge we construct in this work, we’ll get similar complete-knowledge soundness, while making a similar number of $\mathcal{H}$ queries like in this PoCK. The crucial difference is that in our SPoCK the (honest or malicious) prover must make its $\mathcal{H}$ queries *sequentially*. Thus we have a much more restrictive assumption on the adversary,

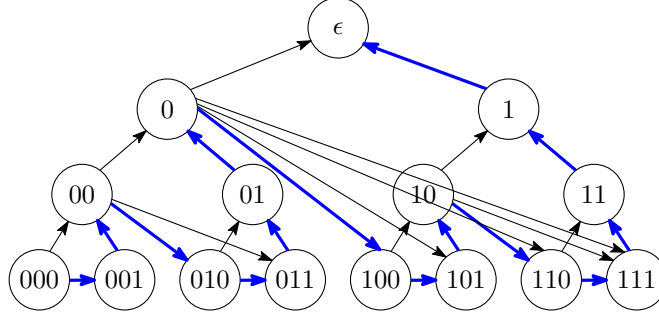


Figure 1: The DAG G_n underlying the PoSW from [CP18]. Here shown for $n = 3$, so we have $N = 2^n$ leaf nodes with labels $\{0, 1\}^n$ and $N' = 2N - 1$ labels $\{0, 1\}^{\leq n}$ in total. The directed path from 0^n to ϵ , indicating the order in which the labels must be computed, is shown in blue.

namely, that it cannot make s encumbered queries *sequentially*, which in practice means we may choose a much smaller s , and thus also get an overall much more efficient construction for a similar level of security.

3 Proofs of Sequential Work

Proofs of sequential work (PoSW) were introduced by Mahmoody, Moran and Vadhan [MMV13], below we consider the more efficient construction of Cohen and Pietrzak [CP18] (but we'll also consider the [MMV13] PoSW in §7 when discussing TEEs). Informally, a PoSW is a proof system by which a prover P convinces a verifier V that they did some inherently sequential computation. The construction from [CP18] is in the random oracle model, and based on a DAG as illustrated in Fig.1. The prover, on input a challenge ρ , uses ρ to salt a random oracle $\mathcal{H}_\rho(\cdot) \stackrel{\text{def}}{=} \mathcal{H}(\rho, \cdot)$, and then sequentially computes a labeling of the DAG, where the label of a node is $\mathcal{H}_\rho$ applied to the labels of its parents. P then sends the label of the root (one can think of this as a Merkle commitment to all the labels) to V , who then chooses M challenges $\gamma = (\gamma_0, \dots, \gamma_{M-1})$ and sends γ to P . The prover P must reply with an opening to the label of the leaf nodes $\gamma_i \in \gamma$.

The security proof in [CP18] goes along the following lines. Let N' be the size of the DAG. If, for some $1 > \alpha > 0$, a malicious prover $\tilde{P}$ only makes $(1 - \alpha)N'$ instead of N' sequential queries (where each query can contain polynomially many inputs to be evaluated in parallel), then the prover will not be able to open an α fraction of the leaf nodes (i.e., for a node $s \in \{0, 1\}^n$, provide the labels of the nodes $\text{open}(s)$ as defined below) and thus fail to convince the verifier with probability $1 - (1 - \alpha)^M$.

For a leaf node $s = s[1 \dots n] \in \{0, 1\}^n$, we denote with $\text{open}(s)$ the nodes whose labels one needs to provide to open the label ℓ_s , that is, the nodes on the

path and co-path from the root to s

$$\text{open}(s) \stackrel{\text{def}}{=} \{s[1 \dots k]\}_{k=0}^n \cup \{s[1 \dots k-1] \parallel (1 - s[k])\}_{k=0}^n \quad (2)$$

4 SPoCK definition

In this paper we construct a public-coin ZK-SPoCK as defined in §4.1 below. For some applications, for example when we want to construct signatures, we want a *non-interactive* zero-knowledge sequential proofs of complete knowledge (NI-ZK-SPoCK). We define those in §4.2, and observe that we can construct a NI-ZK-SPoCK from a public-coin ZK-SPoCK as defined below using the Fiat-Shamir heuristic.

As all our constructions rely on a hash function $\mathcal{H}$, which for the analysis is modeled as a random oracle, we already explicitly model the oracle access to $\mathcal{H}$ in the definition.

4.1 public-coin ZK-SPoCK Definition

A three-move public-coin ZK-SPoCK for a relation $\mathcal{R}$ is given by four algorithms $(\mathbf{P}^{\mathcal{H}} = (\mathbf{P}_1^{\mathcal{H}}, \mathbf{P}_2^{\mathcal{H}}), \mathbf{V}^{\mathcal{H}} = (\mathbf{V}_1^{\mathcal{H}}, \mathbf{V}_2^{\mathcal{H}}), \text{Ext}, \text{Sim}^{\mathcal{H}})$. With $\text{par} = \{\lambda, \Gamma, N, \dots\}$ we denote the public parameters used in the scheme, which must at least include the security parameter λ , a sequentiality parameter $N \in \mathbf{poly}(\lambda)$ and a challenge space Γ .

$\mathbf{P}_1^{\mathcal{H}}(\text{par}, \rho, (x, w)) \rightarrow (\Phi, \text{st})$ on input a salt ρ , an instance with witness $(x, w) \in \mathcal{R}$ and time outputs a message Φ (that is sent to the verifier, think of it as a type of commitment) and a state st (to be passed to the 2nd stage).

$\mathbf{V}_1^{\mathcal{H}}(\text{par}, \rho, x, \Phi) \rightarrow \gamma$ outputs a random challenge $\gamma \leftarrow \Gamma$.

$\mathbf{P}_2^{\mathcal{H}}(\text{par}, \rho, (x, w), \text{st}, \gamma) \rightarrow \pi$ computes the proof from the state and a challenge $\gamma \in \Gamma$.

$\mathbf{V}_2^{\mathcal{H}}(\text{par}, \rho, x, \Phi, \gamma, \pi) \rightarrow \text{acc/rej}$ decides whether to accept or reject.

Ideally, for a given N (which is part of par), we want $\mathbf{P}_1^{\mathcal{H}}(\text{par}, \rho, (x, w))$ to make not much more than N sequential queries to $\mathcal{H}$, while $\mathbf{P}_2^{\mathcal{H}}$ should ideally make $o(N)$ queries and $\mathbf{V}_2^{\mathcal{H}}$ should be highly efficient and run in $\mathbf{poly}(\log(N), \lambda)$ time ($\mathbf{V}_1^{\mathcal{H}}$ is efficient by definition). We require the following properties

Completeness. An honestly generated proof should be accepted. For any $(x, w) \in \mathcal{R}$

$$\Pr \left[\mathbf{V}_2^{\mathcal{H}}(\text{par}, \rho, x, \Phi, \gamma, \pi) = \text{acc} \mid \begin{array}{l} (\Phi, \text{st}) \leftarrow \mathbf{P}_1^{\mathcal{H}}(\text{par}, \rho, (x, w)) \\ \gamma \leftarrow \mathbf{V}_1^{\mathcal{H}}(\text{par}, \rho, x, \Phi) \\ \pi \leftarrow \mathbf{P}_2^{\mathcal{H}}(\text{par}, \rho, (x, w), \text{st}, \gamma) \end{array} \right] = 1. \quad (3)$$

Soundness (Complete-Knowledge) For some parameter s denoting the number of slow queries, and p denoting the number of subadversaries (as discussed in §2.3.3), consider a two part malicious prover $(\tilde{P}_{1,s,p}, \tilde{P}_2)$. $\tilde{P}_{1,s,p}$, apart from the commitment Φ and a state st to be passed to $\tilde{P}_2$, can also choose a partition $\mathcal{Q}_1, \dots, \mathcal{Q}_p$ of all but s of the $\mathcal{H}$ queries it made. We want the probability

$$\Pr_{\rho} \left[\begin{array}{l} V_2^{\mathcal{H}}(\text{par}, \rho, x, \Phi, \gamma, \pi) = \text{acc} \\ \text{and } \forall i \in [p] : (x, w_i) \notin \mathcal{R} \end{array} \middle| \begin{array}{l} (\Phi, \text{st}, \mathcal{Q}_1.. \mathcal{Q}_p) \leftarrow \tilde{P}_{1,s,p}^{\mathcal{H}}(\text{par}, \rho, (x, w)) \\ \gamma \leftarrow V_1^{\mathcal{H}}(\text{par}, \rho, x, \Phi) \\ \pi \leftarrow \tilde{P}_2^{\mathcal{H}}(\text{par}, \rho, (x, w), \text{st}, \gamma) \\ \text{where } \forall i \in [p] : \mathcal{Q}_i \subseteq \mathcal{Q} \\ \text{and } |\cup_{i=1}^p \mathcal{Q}_i| \geq |\mathcal{Q}| - s \\ \forall i \in [p] : w_i \leftarrow \text{Ext}(\text{par}, x, \mathcal{Q}_i) \end{array} \right]$$

that the extractor cannot extract a witness from at least one of the $\mathcal{Q}_i$'s, assuming V_2 accepts, to be negligible.

Soundness (Sequentiality). Consider an adversary $\tilde{P}_1, \tilde{P}_2$, where for some *sequentiality-soundness gap* $\alpha, 0 < \alpha < 1$, $\tilde{P}_1^{\mathcal{H}}$ is allowed at most $N(1 - \alpha)$ sequential queries (where each sequential query can contain polynomially many parallel queries). We want the probability that V_2 accepts

$$\Pr_{\rho} \left[\begin{array}{l} V_2^{\mathcal{H}}(\text{par}, \rho, x, \Phi, \gamma, \pi) = \text{acc} \end{array} \middle| \begin{array}{l} (\Phi, \text{st}) \leftarrow \tilde{P}_1^{\mathcal{H}}(\text{par}, \rho, (x, w)) \\ \gamma \leftarrow V_1^{\mathcal{H}}(\text{par}, \rho, x, \Phi) \\ \pi \leftarrow \tilde{P}_2^{\mathcal{H}}(\text{par}, \rho, (x, w), \text{st}, \gamma) \end{array} \right]$$

to be negligible.

Honest-Verifier Zero-Knowledge. We require the existence of an efficient simulator Sim where for any ρ and $(x, w) \in \mathcal{R}$, $\text{Sim}^{\mathcal{H}}(\text{par}, \rho, x)$ outputs a transcript that is indistinguishable from a transcript (Φ, γ, π) as generated in eq.(3).

4.2 NI-ZK-SPoCK Definition

In this section we define a non-interactive ZK-SPoCK, which can be constructed by using the Fiat-Shamir heuristic to the public-coin ZK-SPoCK from the previous section. Concretely, instead of letting V_1 sample a random challenge γ , this challenge is computed as a hash $\gamma \leftarrow \mathcal{H}(\rho, x, \Phi)$ of the instance and 1st message.

A NI-ZK-SPoCK for a relation $\mathcal{R}$ is given by four algorithms $(P^{\mathcal{H}}, V^{\mathcal{H}}, \text{Ext}, \text{Sim}^{\mathcal{H}})$.

$P^{\mathcal{H}}(\text{par}, \rho, (x, w)) \rightarrow \pi$ on input public parameters par (as in the previous section), a salt ρ , an instance with witness $(x, w) \in \mathcal{R}$ outputs a proof π .

$V^{\mathcal{H}}(\text{par}, \rho, x, \pi) \rightarrow \text{acc/rej}$ decides whether to accept or reject.

We require the following properties

Completeness. An honestly generated proof should be accepted. For any $(x, w) \in \mathcal{R}$

$$\Pr [\mathbf{V}^{\mathcal{H}}(\text{par}, \rho, x, \pi) = \text{acc} \mid \pi \leftarrow \mathbf{P}_1^{\mathcal{H}}(\text{par}, \rho, (x, w))] = 1. \quad (4)$$

Soundness (Complete-Knowledge). For some parameter s denoting the number of slow queries, and p denoting the number of subadversaries (as discussed in §2.3.3), consider a malicious prover $\tilde{\mathbf{P}}$, which apart from the proof π can also choose a partition $\mathcal{Q}_1, \dots, \mathcal{Q}_p$ of all but s of the $\mathcal{H}$ queries it made. We want the probability

$$\Pr_{\rho} \left[\begin{array}{l} \mathbf{V}^{\mathcal{H}}(\text{par}, \rho, x, \pi) = \text{acc} \\ \text{and } \forall i \in [p] : (x, w_i) \notin \mathcal{R} \end{array} \middle| \begin{array}{l} (\pi, \mathcal{Q}_1.. \mathcal{Q}_p) \leftarrow \tilde{\mathbf{P}}_{s,p}^{\mathcal{H}}(\text{par}, \rho, (x, w)) \\ \text{where } \forall i \in [p] : \mathcal{Q}_i \subseteq \mathcal{Q} \\ \text{and } |\cup_{i=1}^p \mathcal{Q}_i| \geq |\mathcal{Q}| - s \\ \forall i \in [p] : w_i \leftarrow \text{Ext}(\text{par}, x, \mathcal{Q}_i) \end{array} \right]$$

that the extractor cannot extract a witness from at least one of the $\mathcal{Q}_i$'s, assuming $\mathbf{V}$ accepts, to be negligible.

Soundness (Sequentiality). Consider an adversary $\tilde{\mathbf{P}}$, where for some *sequentiality-soundness gap* $\alpha, 0 < \alpha < 1$, $\tilde{\mathbf{P}}^{\mathcal{H}}$ is allowed at most $N(1 - \alpha)$ sequential queries (where each sequential query can contain polynomially many parallel queries). We want the probability that $\mathbf{V}$ accepts

$$\Pr_{\rho} [\mathbf{V}^{\mathcal{H}}(\text{par}, \rho, x, \pi) = \text{acc} \mid \pi \leftarrow \tilde{\mathbf{P}}^{\mathcal{H}}(\text{par}, \rho, (x, w))]$$

to be negligible.

Zero-Knowledge. We require the existence of an efficient simulator Sim where for any ρ and $(x, w) \in \mathcal{R}$, $\text{Sim}^{\mathcal{H}}(\text{par}, \rho, x)$ outputs a proof π that is indistinguishable from a real proof $\pi \leftarrow \mathbf{P}^{\mathcal{H}}(\text{par}, \rho, (x, w))$. The simulator is not given a witness, but instead is allowed to program the random oracle $\mathcal{H}$.

5 Construction of a public-coin ZK-SPoCK

In this section we present our sequential proof of complete knowledge. Figure 2 illustrates this construction which is based on the DAG G_n from the [CP18] PoSW as shown in Figure 1.

The construction relies on the following building blocks and parameters.

Σ A Sigma protocol $(\mathbf{P}_{\Sigma}, \mathbf{V}_{\Sigma}, \text{Ext}_{\Sigma}, \text{Sim}_{\Sigma})$ for a relation $\mathcal{R}$.

$\mathcal{H}, \lambda, \rho$ A hash function (modeled as random oracle in the analysis) $\mathcal{H} : \{0, 1\}^* \rightarrow \{0, 1\}^{\lambda}$. Here λ is a statistical security parameter, say $\lambda = 256$. The salt ρ is used as a prefix to $\mathcal{H}$, we use the notation $\mathcal{H}_{\rho}(\cdot) = \mathcal{H}(\rho, \cdot)$.

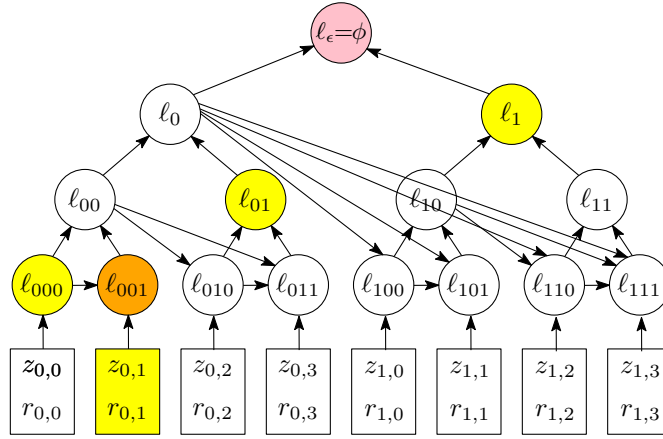


Figure 2: Illustration of our ZK-SPoCK for a toy example with $M = 2$ and $N = 8$. The prover will sample $\mathbf{a} = (a_0, a_1)$ and $z_{i,j}$ that satisfy $\mathbf{V}_{\Sigma}(a_i, j, z_{i,j}) \rightarrow \text{acc}$ and (pseudo)random $r_{i,j}$ values. It will then compute all labels sequentially, where a label is the hash of the labels of its parents (and also $\mathbf{a}$ and a beacon ρ). The commitment $\phi = \ell_{\epsilon}$ and $\mathbf{a}$ are sent to the verifier, who sends a challenge $\gamma = (\gamma_0, \gamma_1) \in [0, N/M - 1]^M$, say $(1, 2)$, and the prover must now open ℓ_{001} and ℓ_{110} (the labels above $z_{i,\gamma_j}, i \in [0, N/M - 1]$). To open the (orange) label ℓ_{001} , we must provide all the (yellow) labels on the co-path from node 001 to ϵ (i.e., $\ell_{000}, \ell_{01}, \ell_1$), together with the $z_{0,1}$ value.

$G_n, N = 2^n, N' = 2N - 1$ The DAG from [CP18] on $N' = 2N - 1 = 2^{n+1} - 1$ vertices with N leafs. The honest prover must make N' sequential $\mathcal{H}$ queries.

$M = 2^m$ The challenge space in our construction is $\Gamma \stackrel{\text{def}}{=} [0, N/M]^M$. On challenge $\gamma = (\gamma_0, \dots, \gamma_{M-1}) \leftarrow \Gamma$, the prover must open M leaf labels. The parameter M will determine what sequentiality-soundness gap α we can tolerate: a malicious prover making $N(1 - \alpha)$ sequential queries, will succeed in breaking sequentiality soundness with probability $(1 - \alpha)^M + \text{negl}(\lambda)$. We will assume that M is a power of two as this simplifies exposition, and set $m = \log(M)$.

o The parameter $o \in [0, n - 1]$ is only used for efficiency during the opening phase: by storing the labels of the top o levels in the tree (which requires $\lambda \cdot 2^{o+1}$ bits), one can recompute the opening of a leaf label making 2^{n-o+1} $\mathcal{H}$ queries.

Our three-move public-coin SPoCK $(P_1^{\mathcal{H}}, P_2^{\mathcal{H}}, V_1^{\mathcal{H}}, V_2^{\mathcal{H}}, \text{Ext}, \text{Sim}^{\mathcal{H}})$ is defined below with public parameters $\text{par} = (\lambda, n, m, o)$ as outlined above. It will be convenient to define a mapping $\Lambda : \{0, 1\}^n \rightarrow [0, M - 1] \times [0, N/M - 1]$ from leaf labels $\{0, 1\}^n$ to indices (below think of $s \in \{0, 1\}^n$ as a number in $[0, 2^n - 1]$).

$$\Lambda(s) \stackrel{\text{def}}{=} (\lfloor s/2^{n-m} \rfloor, s \bmod 2^{n-m}) \quad (5)$$

$P_1^{\mathcal{H}}(\text{par}, \rho, (x, w)) \rightarrow ((\mathbf{a}, \phi), \text{st})$:

- Sample M first messages $\mathbf{a} = (a_0, \dots, a_{M-1})$, $(a_i, \text{st}_i) \leftarrow P_{\Sigma}(x, w)$ of the Σ protocol.
- For each $i \in [0, 2^m - 1]$, $j \in [0, 2^{n-m} - 1]$ compute the third message of a Σ transcript $(a_i, j, z_{i,j})$

$$z_{i,j} \leftarrow P_{\Sigma}((x, w), j, \text{st}_i). \quad (6)$$

- Sample a random $r \leftarrow \{0, 1\}^{\lambda}$ and define

$$r_{i,j} := \mathcal{H}(r, i, j). \quad (7)$$

Think of the $r_{i,j}$ as (pseudo)random values which can be derived from the short seed r .

- Compute a labelling $\{\ell_s\}_{s \in \{0,1\}^{\leq n}}$ of G_n (making N sequential queries to $\mathcal{H}_{\rho}$) as

$$\ell_s := H_{\rho, \mathbf{a}}(s, \epsilon_s, \ell_{p_1}, \dots, \ell_{p_d}), \quad (8)$$

where $(p_1, \dots, p_d) = \text{parents}(s)$ and

1. If s is not a leaf, i.e., $s \in \{0, 1\}^{<n}$, then $\epsilon_s = \emptyset$.
2. If s is a leaf, let $(i, j) := \Lambda(s)$ (cf. eq.(5)) and set $\epsilon_s = (z_{i,j}, r_{i,j})$.

Let $\phi = \ell_\epsilon$ be the label of the root (which we think of as Merkle commitment to all labels). To speed up openings in the 2nd phase, we'll also store the labels in the o highest layers. $\mathbf{st} = (\{\ell_s\}_{s \in \{0,1\}^{\leq o}}, r)$.

$V_1^{\mathcal{H}}(\text{par}, \rho, x, \Phi) \rightarrow \gamma$: Outputs $\gamma = (\gamma_0, \dots, \gamma_{M-1}) \leftarrow^* \Gamma \stackrel{\text{def}}{=} [0, N/M]^M$.

$P_2^{\mathcal{H}}(\text{par}, \rho, (x, w), \mathbf{st}, \gamma) \rightarrow \tau$: Parse $\mathbf{st} = \{\ell_s\}_{s \in \{0,1\}^{\leq m}}$ and $\gamma = (\gamma_0, \dots, \gamma_{M-1})$. Recompute and output the openings of the labels $s_i \stackrel{\text{def}}{=} \Lambda^{-1}(i, \gamma_i)$ for $i \in [0, M-1]$, that is (with **open** as in eq.(2), $z_{i,j}$ as in eq.(6) and $r_{i,j}$ as in eq.(7))

$$\tau = (\tau_0, \dots, \tau_{M-1}) \text{ where } \tau_i = (\{\ell_u\}_{u \in \text{open}(s_i)}, z_{i,\gamma_i}, r_{i,\gamma_i}), \quad (9)$$

i.e., τ_i is the opening of the commitment $\phi = \ell_\epsilon$ on the positions of the committed labels of $s_i \stackrel{\text{def}}{=} \Lambda^{-1}(i, \gamma_i)$ for $i \in [0, M-1]$.

$V_2^{\mathcal{H}}(\text{par}, \rho, x, (\mathbf{a}, \phi), \gamma, \tau) \rightarrow \text{acc/rej}$: parse the input into the public parameters par , the beacon ρ , the statement x , the commitment $(\mathbf{a} = (a_0, \dots, a_{M-1}), \phi)$, the challenge $\gamma = (\gamma_0, \dots, \gamma_{M-1})$ and opening $\tau = (\tau_0, \dots, \tau_{M-1})$ where $\tau_i = (\{\ell_u\}_{u \in \text{open}(s_i)}, z_{i,\gamma_i}, r_{i,\gamma_i})$ with $s_i = \Lambda^{-1}(i, \gamma_i)$. Now check for all $i \in [0, M-1]$ that

1. $V_\Sigma(a_i, \gamma_i, z_{i,\gamma_i}) \stackrel{?}{=} \text{acc}$ is an accepting transcript of the Σ protocol using the committed a_i as the first, and challenge γ_i as the second message.
2. τ_i is a correct opening (to the label ℓ_{s_i} and transcript $(a_i, \gamma_i, z_{i,\gamma_i})$) by recomputing the labels as in eq.(8).

The first check above will be relevant when we argue complete-knowledge soundness, while the second check is relevant to argue sequentiality soundness.

6 Analysis of the ZK-SPoCK

In this section we analyze the efficiency and prove the soundness and zero-knowledge properties of the public-coin ZK-SPoCK from §5.

6.1 Efficiency

Our scheme is basically as efficient as the underlying PoSW from [CP18]. The additional computation and communication one needs to additionally embed and check Σ protocol tuples is small in comparison as we'll outline below.

Compared to [CP18], where we use only ρ as a prefix for hashing, here (cf. eq.(8)) we additionally use $\mathbf{a}$ in the prefix. This is not an issue even though $\mathbf{a}$ is somewhat long, we use the same prefix throughout the entire labelling. So, we

can first hash the prefix $(\rho, \mathbf{a})$ into as short (say, 256 bits) prefix ρ' , and then use $\mathcal{H}_{\rho'}(\cdot) = \mathcal{H}(\rho', \cdot)$ to compute labels.

Another difference is that we additionally hash the random $r_{i,j}$ with the leaf labels. Those are fairly short, moreover we derive them from a single seed r , so there's no extra storage requirement on the prover, it just needs to store the short seed r .

Finally, the prover needs to compute all the $z_{i,j}$. While computing the third message in a Σ protocol can be expensive, we note that we only need to compute sequential tuples, i.e., $(a_i, 0, z_{i,0}), (a_i, 1, z_{i,1}), (a_i, 2, z_{i,2}), \dots$. At least in Schnorr's protocol (cf. eq.(1)) for discrete logarithms, going from $z_{i,j}$ to $z_{i,j+1}$ is very efficient and it just requires a single addition, i.e., $z_{i,j+1} = z_{i,j} + w$.

6.2 Soundness (Sequentiality)

To argue sequentiality, we will ignore the $r_{i,j}, z_{i,j}$ which are part of the $\mathcal{H}$ input when computing the label of a leaf node in G_n , then we get basically the PoSW from [CP18]. The only difference is that in [CP18] the prover must open some M randomly chosen leaf nodes, while we split the N leaf nodes into M “buckets”, and the prover must open one random leave from each bucket (recall that the reason for this is simply that we want to open exactly one transcript for any first message a_i).

We will not repeat the entire proof from [CP18] here, but only explain how their analysis goes through with the different distribution of challenges as just mentioned. Let us mention that we actually slightly improve on [CP18] by using a different distribution, using the same distribution in [CP18] would result in the same security, but would slightly reduce the length of the challenge message $(\gamma_0, \dots, \gamma_{M-1})$ from $m \cdot n$ to $m \cdot (n - m)$ bits. Theorem 1 from [CP18] (adapted to the notation in this paper) states that

Lemma 1 (Sequentiality Soundness, basically Thm. 1 from [CP18]). *Consider the PoSW from §5, with parameters $\lambda, M = 2^m, N = 2^n, N' = 2N - 1$ and a “soundness gap” $\alpha > 0$. Consider a malicious prover $(\tilde{P}_1, \tilde{P}_2)$ where $\tilde{P}_1$ makes at most $(1 - \alpha)N'$ sequential queries to $\mathcal{H}$, and at most q queries in total, then $V_2^{\mathcal{H}}$ will output rej with probability*

$$1 - (1 - \alpha)^M - \frac{N' \cdot \lambda \cdot q^2}{2^\lambda}.$$

The term $N \cdot \lambda \cdot q^2 / 2^\lambda$, which is exponentially small in the security parameter, basically accounts for the probability that $\tilde{P}_1$ or $\tilde{P}_2$ makes a “surprising” $\mathcal{H}$ query, which is a collision or where we observe an output that has previously been used as part of an input to $\mathcal{H}$.

Assuming such a bad query never happens, we can argue (cf., [CP18] for the details) that the prover, who made only $N'(1 - \alpha)$ sequential queries, must fail to open an α fraction of the leaf labels. The probability that it will succeed on all M randomly chosen leaf labels is thus at most $(1 - \alpha)^M$.

We can think of this as an experiment where we have N balls, $\alpha \cdot N$ of which are bad. We must pick M balls (with replacement), and want to know the probability that none of those is bad. In our construction, we have M bins, each has N/M balls with a *total* of $\alpha \cdot N$ bad balls. We pick one ball from each bin, and want an upper bound on the probability that none is bad (and thus $\tilde{P}_1, \tilde{P}_2$ succeeds in convincing V_2). Let $B_i, \sum_{i=0}^{M-1} B_i = \alpha \cdot N$ denote the number of bad balls in bin i . We will need the following classical inequality

Proposition 1 (Arithmetic Mean-Geometric Mean (AM-GM) inequality). *Let $x_1, \dots, x_t \in \mathbb{R}^{\geq 0}$. Then*

$$\left(\prod_{i=1}^t x_i \right)^{\frac{1}{t}} \leq \frac{\sum_{i=1}^t x_i}{t}$$

Using this in the third step below, we get

$$\begin{aligned} \Pr[V_2 \text{ accepts}] &= \Pr[\text{all } M \text{ balls are good}] \\ &= \prod_{i=0}^{M-1} \left(1 - \frac{B_i}{N/M} \right) \\ &\leq \left(\sum_{i=0}^{M-1} \frac{1 - \frac{B_i}{N/M}}{M} \right)^M \\ &= \left(1 - \frac{\sum_{i=0}^{M-1} B_i}{N/M \cdot M} \right)^M \\ &= \left(1 - \frac{\alpha \cdot N}{N} \right)^M \\ &= (1 - \alpha)^M. \end{aligned}$$

6.3 Soundness (Complete-Knowledge)

To prove complete-knowledge soundness §4.1 of our construction from §5 we must show that the probability below is negligible.

Lemma 2 (Complete-Knowledge Soundness). *Consider a malicious prover $\tilde{P}_{1,s,p}, \tilde{P}_2$ (as in §4.1) and let q denote the number of $\mathcal{H}$ queries made by $\tilde{P}_{1,s,p}$ and $\tilde{P}_2$. The complete-knowledge soundness for the construction from §4.1 is*

$$\Pr_{\rho} \left[\begin{array}{l} V_2^{\mathcal{H}}(\text{par}, \rho, x, (\phi, \mathbf{a}), \gamma, \tau) = \text{acc} \\ \text{and } \forall i \in [p] : (x, w_i) \notin \mathcal{R} \end{array} \middle| \begin{array}{l} ((\mathbf{a}, \phi), \text{st}, \mathcal{Q}_1.. \mathcal{Q}_p) \leftarrow \tilde{P}_{1,s,p}^{\mathcal{H}}(\text{par}, \rho, (x, w)) \\ \gamma \leftarrow \text{\$ } [0, N/M]^M \\ \tau \leftarrow \tilde{P}_2^{\mathcal{H}}(\text{st}, \text{par}, \rho, (x, w), \text{st}, \gamma) \\ \text{where } \forall i \in [p] : \mathcal{Q}_i \subseteq \mathcal{Q} \\ \text{and } |\cup_{i=1}^n \mathcal{Q}_i| \geq |\mathcal{Q}| - s \\ \forall i \in [p] : w_i \leftarrow \text{Ext}(\text{par}, x, \mathcal{Q}_i) \end{array} \right]$$

$$\leq \left(\frac{s + M \cdot p}{N} \right)^M + q^2/2^\lambda$$

Proof. Except with exponentially small probability $q^2/2^\lambda$, $\tilde{P}_2$ will only be able to open the $z_{i,j}$ values as in eq.(9) which were used, and in particular queried, by $\tilde{P}_{1,s,p}$ when computing the commitment ϕ . Let Z denote the “good” such $z_{i,j}$ ’s (i.e., the ones $\tilde{P}_2$ can actually open) where $(a_i, j, z_{i,j})$ is a valid triple.

Assume that $|Z| > s + M \cdot p$. Then, by the pigeonhole principle, at least one of the Q_i ’s must contain two $z_{i,j}, z_{i,j'} \in Z$, and by special soundness, for this Q_i , $w_i \leftarrow \text{Ext}(\text{par}, x, Q_i)$ will be a valid witness (and thus the probability above is zero). So now assume $|Z| \leq s + M \cdot p$. The probability that all the M challenges open to valid $z_{i,j}$ ’s is at most

$$\left(\frac{|Z|}{N} \right)^M \leq \left(\frac{s + M \cdot p}{N} \right)^M. \quad (10)$$

The bound above is easy to see assuming $\tilde{P}_2$ must open M random leaf labels. It’s simply the probability that all M balls we pick (with repetition) at random from a bin with N balls are good, assuming the bin contains $|Z|$ good balls. As in the previous section, using the AM-GM inequality, this bound also holds for our distribution (where we consider M bins, each containing N/M balls with a total of $|Z|$ good balls, and we pick one ball at random from each bin). $\square$

Numerical example. Let $s' = s + M \cdot p$, as in practice $M \cdot p \ll s$, just think of s' as the number of slow queries. Consider parameters $s' = 2^{30}$, $M = 2^4 = 16$, $N = 2^{37}$, which roughly correspond to the parameters we chose in the numerical example for the Fischlin PoCK (note that $N' = 2N - 1 = 2^{38} - 1$ is the number of hash queries required by the prover, while in §2.3.4 this was $M \cdot 2^d = 2^{38}$). The bound in eq.(10) becomes $(2^{-30}/2^{-37})^{16} = 2^{-112}$, thus similar to the 2^{-116} we got for the PoCK in §2.3.4.

But as discussed in §2.3.4, the real advantage comes from the fact that in practice we can choose a much smaller s for a SPoCK than for a PoCK, as in the former s corresponds to *sequential* encumbered queries, so overall we expect this SPoCK be much more efficient for a similar security level.

6.4 Zero-Knowledge

In this section we will prove the zero-knowledge property of the ZK-SPoCK from §5.

Theorem 1. *The ZK-SPoCK from §5 satisfies honest-verifier zero-knowledge property in the random oracle model (as defined in §4.1).*

Proof. We must construct a simulator $\text{Sim}^{\mathcal{H}}(\text{par}, \rho, x)$ that outputs a transcript $((\phi, \mathbf{a}), \gamma, \tau)$ that is indistinguishable from a real transcript, this is done as follows:

1. Sample a random challenge $\gamma = (\gamma_0, \dots, \gamma_{M-1}) \leftarrow [0, N/M]^M$
2. Using the simulator for the Σ protocol, for $i \in [0, M-1]$ sample tuples $(a_i, \gamma_i, z_{i,\gamma_i})$ and set $\mathbf{a} := (a_0, \dots, a_{M-1})$.
3. For all i and $j \neq \gamma_i$, pick $z_{i,j} \leftarrow_{\$} \{0,1\}^\lambda$ at random.
4. For all i, j pick $r_{i,j} \leftarrow_{\$} \{0,1\}^\lambda$ at random.
5. Using the values above, compute all labels as in eq.(8).
6. Using the values computed so far, define τ as in eq.(9).
7. Output $((\phi, \mathbf{a}), \gamma, \tau)$

□

This gives a valid transcript by construction, i.e., $V_2^{\mathcal{H}}(\text{par}, \rho, x, (\mathbf{a}, \phi), \gamma, \tau) = \text{acc}$. But this transcript is of course not correctly computed, as for all $(i, j), j \neq \gamma_i$, the labels of the leaf nodes are computing using a wrong $z_{i,j}$, i.e., $(a_i, j, z_{i,j})$ will be rejected by V_Σ . But even a distinguisher that knows the witness (but makes $\ll 2^\lambda$ queries to $\mathcal{H}$) will not be able to distinguish this simulated from a real transcript. The reason is that, while such a distinguisher can compute the correct $z_{i,j}$, it will not be able to verify if the leaf label that used $z_{i,j}$ as input was computed correctly, as this input also includes the random value $r_{i,j} \in \{0,1\}^\lambda$.

7 TEEs

The SPoCK constructed in this paper requires a long sequential computation, which makes it hard to encumber when using MPC, unfortunately it's providing less security when considering the other main mechanism to encumber a computation, namely using trusted execution environments (TEE) like Intel's SGX enclaves, as those contain fairly fast cores.

On the other hand, trusted execution environments (TEEs) typically provide only a limited amount of protected memory, and accessing data beyond this region requires secure paging to untrusted memory, incurring substantial overhead. For instance, Intel SGX enclaves are restricted to a relatively small protected memory region (typically 64 to 256MB), and when the working set exceeds this size, encrypted paging leads to significant performance degradation. Prior work reports slowdowns of several times for memory-intensive workloads once paging is triggered [TSB18]. Consequently, functions that require large working memory together with frequent accesses to pseudorandom locations are particularly inefficient to evaluate inside TEEs, as such access patterns defeat caching and cause repeated secure page swaps.

Or SPoCK is based on the PoSW from [CP18], which is based on the DAG as illustrated in Figure 1. This DAG has the property that its labelling can be computed using little memory: $\text{lust } \log(N)$ labels must be stored at any point. This was a massive improvement of the original construction of a PoSW from

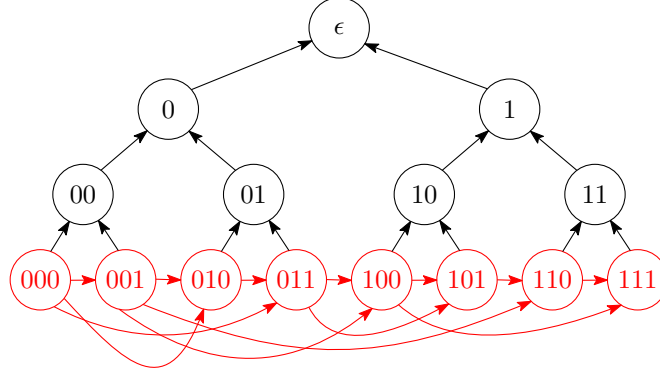


Figure 3: The graph underlying the [MMV13] PoSW. A depth robust DAG (in red) with a full binary tree on top.

[MMV13], which was based on depth-robust graphs, and where the prover had to store most of the up to N' labels when computing the labelling. Interestingly, while this was a major disadvantage of their PoSW, depth-robust graphs later turned out to be the right tool to construct cryptographic primitives that provably require a lot of space or memory, like proofs of space [DFKP15] or memory-hard functions [ABP17].

We can easily adapt our SPoCK to use the [MMV13] PoSW instead of [CP18]. For this, we would start with a depth-robust DAG on N nodes $\{0, 1\}^n$, and then put a complete-binary tree (as a DAG) on top of it as illustrated in Fig. 3. To compute the labelling of this DAG one first (sequentially) computes the N label of the depth-robust DAG, and then (potentially in parallel) computes the labels of the tree above (as in our construction, the label of the root now commits all the labels below).

The “space-hard” SPoCK constructed this way will inherit all the security properties of our construction, but it will now put a higher burden on (honest or malicious) provers, who must now use a large amount $\Theta(N \cdot \lambda)$ of memory. If this is too much for the honest prover, one can e.g. concatenate T depth robust graphs on N/T nodes each instead of using one big DAG on N nodes, this will then require in the order of $N \cdot \lambda/T$ space.

8 Conclusions

In this work we showed how proofs of complete knowledge, which prevent encumbrance of the secret by forcing the prover to make a huge number of oracle queries, can be combined with techniques from timed cryptography, and (in §7) also space hard cryptography, to significantly harden the scheme against encumbrance, as now the queries must be done sequentially (which is really hard to do in MPC), and can additionally require a large memory with lots of frequent random accesses (which is hard for TEEs).

These ideas can likely be also applied to other primitives from individual cryptography. Our work opens the possibility that such primitives can be implemented on standard hardware, rather than relying on special hardware like Bitcoin ASICs like the existing constructions.

Acknowledgements. This research was funded in whole or in part by the Austrian Science Fund (FWF) 10.55776/F85.

References

- [ABP17] Joël Alwen, Jeremiah Blocki, and Krzysztof Pietrzak. Depth-robust graphs and their cumulative memory complexity. In Jean-Sébastien Coron and Jesper Buus Nielsen, editors, *Advances in Cryptology – EUROCRYPT 2017, Part III*, volume 10212 of *Lecture Notes in Computer Science*, pages 3–32, Paris, France, April 30 – May 4, 2017. Springer, Cham, Switzerland. 20
- [AFK⁺25] James Austgen, Andrés Fábrega, Mahimna Kelkar, Dani Vilardell, Sarah Allen, Kushal Babel, Jay Yu, and Ari Juels. Liquefaction: Privately liquefying blockchain assets. In Marina Blanton, William Enck, and Cristina Nita-Rotaru, editors, *IEEE Symposium on Security and Privacy, SP 2025, San Francisco, CA, USA, May 12-15, 2025*, pages 1493–1511. IEEE, 2025. 3
- [BCCT13] Nir Bitansky, Ran Canetti, Alessandro Chiesa, and Eran Tromer. Recursive composition and bootstrapping for SNARKS and proof-carrying data. In Dan Boneh, Tim Roughgarden, and Joan Feigenbaum, editors, *45th Annual ACM Symposium on Theory of Computing*, pages 111–120, Palo Alto, CA, USA, June 1–4, 2013. ACM Press. 2
- [BDF⁺25] Jan Bormet, Stefan Dziembowski, Sebastian Faust, Tomasz Lazurek, and Marcin Mielniczuk. Strong secret sharing with snitching. In Yael Tauman Kalai and Seny F. Kamara, editors, *Advances in Cryptology – CRYPTO 2025, Part IV*, volume 16003 of *Lecture Notes in Computer Science*, pages 295–327, Santa Barbara, CA, USA, August 17–21, 2025. Springer, Cham, Switzerland. 3
- [Can01] Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *42nd Annual Symposium on Foundations of Computer Science*, pages 136–145, Las Vegas, NV, USA, October 14–17, 2001. IEEE Computer Society Press. 2
- [CP18] Bram Cohen and Krzysztof Pietrzak. Simple proofs of sequential work. In Jesper Buus Nielsen and Vincent Rijmen, editors, *Advances in Cryptology – EUROCRYPT 2018, Part II*, volume 10821

- of *Lecture Notes in Computer Science*, pages 451–467, Tel Aviv, Israel, April 29 – May 3, 2018. Springer, Cham, Switzerland. [4](#), [9](#), [12](#), [14](#), [15](#), [16](#), [19](#), [20](#)
- [DFKP15] Stefan Dziembowski, Sebastian Faust, Vladimir Kolmogorov, and Krzysztof Pietrzak. Proofs of space. In Rosario Gennaro and Matthew J. B. Robshaw, editors, *Advances in Cryptology – CRYPTO 2015, Part II*, volume 9216 of *Lecture Notes in Computer Science*, pages 585–605, Santa Barbara, CA, USA, August 16–20, 2015. Springer Berlin Heidelberg, Germany. [20](#)
- [DFL23] Stefan Dziembowski, Sebastian Faust, and Tomasz Lazurek. Individual cryptography. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023, Part II*, volume 14082 of *Lecture Notes in Computer Science*, pages 547–579, Santa Barbara, CA, USA, August 20–24, 2023. Springer, Cham, Switzerland. [3](#)
- [DFLM24] Stefan Dziembowski, Sebastian Faust, Tomasz Lazurek, and Marcin Mielniczuk. Secret sharing with snitching. In Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie, editors, *ACM CCS 2024: 31st Conference on Computer and Communications Security*, pages 840–853, Salt Lake City, UT, USA, October 14–18, 2024. ACM Press. [3](#)
- [DGK17] Yevgeniy Dodis, Siyao Guo, and Jonathan Katz. Fixing cracks in the concrete: Random oracles with auxiliary input, revisited. In Jean-Sébastien Coron and Jesper Buus Nielsen, editors, *Advances in Cryptology – EUROCRYPT 2017, Part II*, volume 10211 of *Lecture Notes in Computer Science*, pages 473–495, Paris, France, April 30 – May 4, 2017. Springer, Cham, Switzerland. [7](#)
- [FFS87] Uriel Feige, Amos Fiat, and Adi Shamir. Zero knowledge proofs of identity. In Alfred Aho, editor, *19th Annual ACM Symposium on Theory of Computing*, pages 210–217, New York City, NY, USA, May 25–27, 1987. ACM Press. [2](#), [4](#)
- [Fis05] Marc Fischlin. Communication-efficient non-interactive proofs of knowledge with online extractors. In Victor Shoup, editor, *Advances in Cryptology – CRYPTO 2005*, volume 3621 of *Lecture Notes in Computer Science*, pages 152–168, Santa Barbara, CA, USA, August 14–18, 2005. Springer Berlin Heidelberg, Germany. [2](#), [6](#), [7](#)
- [FS87] Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In Andrew M. Odlyzko, editor, *Advances in Cryptology – CRYPTO’86*, volume 263 of *Lecture Notes in Computer Science*, pages 186–194, Santa Barbara, CA, USA, August 1987. Springer Berlin Heidelberg, Germany. [2](#), [4](#), [5](#)

- [GMR85] Shafi Goldwasser, Silvio Micali, and Charles Rackoff. The knowledge complexity of interactive proof-systems (extended abstract). In *17th Annual ACM Symposium on Theory of Computing*, pages 291–304, Providence, RI, USA, May 6–8, 1985. ACM Press. [2](#)
- [GMW86] Oded Goldreich, Silvio Micali, and Avi Wigderson. Proofs that yield nothing but their validity and a methodology of cryptographic protocol design (extended abstract). In *27th Annual Symposium on Foundations of Computer Science*, pages 174–187, Toronto, Ontario, Canada, October 27–29, 1986. IEEE Computer Society Press. [2](#), [4](#)
- [KBD⁺24] Mahimna Kelkar, Kushal Babel, Philip Daian, James Austgen, Vitalik Buterin, and Ari Juels. Complete knowledge: Preventing encumbrance of cryptographic secrets. In Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie, editors, *ACM CCS 2024: 31st Conference on Computer and Communications Security*, pages 2415–2429, Salt Lake City, UT, USA, October 14–18, 2024. ACM Press. [3](#), [4](#)
- [KGP⁺25] Mahimna Kelkar, Aadityan Ganesh, Aditi Partap, Joseph Bonneau, and S. Matthew Weinberg. Breaking omertà: On threshold cryptography, smart collusion, and whistleblowing. In Chun-Ying Huang, Jyh-Cheng Chen, Shiuh-Pyng Shieh, David Lie, and Véronique Cortier, editors, *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security, CCS 2025, Taipei, Taiwan, October 13-17, 2025*, pages 3505–3519. ACM, 2025. [3](#)
- [MMV13] Mohammad Mahmoody, Tal Moran, and Salil P. Vadhan. Publicly verifiable proofs of sequential work. In Robert D. Kleinberg, editor, *ITCS 2013: 4th Innovations in Theoretical Computer Science*, pages 373–388, Berkeley, CA, USA, January 9–12, 2013. Association for Computing Machinery. [4](#), [9](#), [20](#)
- [Pas03] Rafael Pass. On deniability in the common reference string and random oracle model. In Dan Boneh, editor, *Advances in Cryptology – CRYPTO 2003*, volume 2729 of *Lecture Notes in Computer Science*, pages 316–337, Santa Barbara, CA, USA, August 17–21, 2003. Springer Berlin Heidelberg, Germany. [2](#)
- [TSB18] Meysam Taassori, Ali Shafiee, and Rajeev Balasubramonian. VAULT: reducing paging overheads in SGX with efficient integrity verification structures. In Xipeng Shen, James Tuck, Ricardo Bianchini, and Vivek Sarkar, editors, *Proceedings of the Twenty-Third International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS 2018, Williamsburg, VA, USA, March 24-28, 2018*, pages 665–678. ACM, 2018. [19](#)
- [Unr12] Dominique Unruh. Quantum proofs of knowledge. In David Pointcheval and Thomas Johansson, editors, *Advances in Cryptology*

– *EUROCRYPT 2012*, volume 7237 of *Lecture Notes in Computer Science*, pages 135–152, Cambridge, UK, April 15–19, 2012. Springer Berlin Heidelberg, Germany. [2](#)